2주차 읽기자료: 정상상태 MLP 학습과 평가

1. CSTR의 세 성분 정상상태 출력

1차 연속반응 A → B → C의 시간 미분을 각각 0으로 놓고 A, B, C 순서로 정상 농도를 구합니다.

$$C_A = \frac{C_{\mathrm{Af}}}{1 + k_1\tau}$$

$$C_B = \frac{k_1\tau C_{\mathrm{Af}}}{(1 + k_1\tau)(1 + k_2\tau)}$$

$$C_C = k_2\tau C_B$$

330 K에서 첫 번째와 두 번째 반응의 속도상수는 0.2/min, 0.1/min입니다. 체류시간 5 min·유입 농도 1.2 mol/L에서 농도는 [0.6, 0.4, 0.2] mol/L이고 합은 1.2입니다. 이 세 값이 정상상태 학습 데이터 한 행의 기준 출력입니다.

입력 [T, τ, C_Af]와 출력 [C_A, C_B, C_C]의 한 쌍은 하나의 정상상태 운전 조건에 대응합니다.

2. 학습 전에 파라미터 수 계산하기

완전연결층(dense layer)은 입력·출력 쌍마다 가중치 하나, 출력마다 bias 하나를 둡니다. 정상상태 MLP의 구조는 3 → 32 → 32 → 3입니다.

Layer	개수
첫 hidden layer	3 × 32 + 32 = 128
두 번째 hidden layer	32 × 32 + 32 = 1056
Output layer	32 × 3 + 3 = 99
합계	**1283**

$$h_1 = \mathrm{ReLU}\left(W_1 \tilde{\mathrm{x}} + b_1\right)$$
$$h_2 = \mathrm{ReLU}\left(W_2 h_1 + b_2\right)$$
$$\hat{z} = W_3 h_2 + b_3$$

ReLU는 은닉층의 활성화 전 값마다 max(0, 값)을 적용합니다. 선형 출력층은 음수 농도도 반환할 수 있으므로 비음수성과 농도 합의 일관성을 따로 평가해야 합니다.

열벡터 수식을 batch 행렬로 계산하기

수식은 입력을 열벡터로 씁니다. Batch 코드가 입력을 (N,3)의 행으로 저장하면 첫 층은 `X @ W1.T + b1`이고 결과는 (N,32)입니다. 전치는 같은 가중합을 행 단위 저장 방식에 맞추며, bias 32개는 각 데이터 행에 반복 적용됩니다.

배열	크기	각 축의 의미
X	N×3	데이터 행, 입력 변수
W1.T	3×32	입력 변수, 은닉 뉴런
X @ W1.T	N×32	데이터 행, 은닉 뉴런
b1	32	각 뉴런의 bias

두 입력 행 [1,0,−1], [0,1,1]과 첫 뉴런 가중치 [1,0,0], bias 0.2를 생각합니다. 첫 출력 열은 각각 1×1+0×0−1×0+0.2=1.2와 0×1+1×0+1×0+0.2=0.2입니다. 나머지 31개 열도 해당 뉴런의 가중치로 같은 계산을 합니다.

행렬곱 뒤 ReLU는 각 원소에 따로 적용합니다. 행 하나가 다른 데이터 행과 섞이는 연산은 아닙니다.

3. 출력 scaling과 성분별 loss 가중치

입력 scaling에는 명시한 bounds를 사용합니다. 출력 표준화의 평균과 표준편차는 train 데이터에서만 계산하며, 농도 오차를 보고할 때는 물리 단위로 복원합니다.

$$\tilde{\mathrm{x}}_j = 2\frac{x_j - l_j}{u_j - l_j} - 1$$

$$z_j = \frac{y_j - \mu_j}{s_j}, \quad \hat{y}_j = \mu_j + s_j \hat{z}_j$$

$$L = \frac{1}{3N} \sum_{n=1}^{N} \sum_{j=1}^{3} \left(\hat{z}_{n,j} - z_{n,j} \right)^2$$

원래 단위의 제곱 오차에는 train 표준편차 제곱의 역수가 가중치로 붙습니다. A의 표준편차가 약 0.283, B가 0.110이면 같은 제곱 오차에 B는 A보다 약 6.62배의 가중치를 받습니다.

성분별 test RMSE와 농도 합 residual로 출력 scaling 전후를 비교합니다. 평균 방식은 PyTorch 공식 MSE 정의에서 확인합니다.

4. 역전파와 파라미터 갱신 예제

3 → 2 → 3 신경망의 초기값은 `W1 = [[1, 0, 0], [0, 0, 1]]`, `W2 = [[1, 0], [0, 1], [1, 1]]` 이며 bias는 모두 0입니다. 입력 [1, 0, −1]의 은닉층 활성화 전 값은 [1, −1], ReLU 출력은 [1, 0], 예측은 [1, 0, 1]입니다. 기준값 [0, 1, 0]에 대한 오차는 [1, −1, 1]입니다.

$$\delta_2 = e, \nabla_{W_2} L = eh^T = \begin{bmatrix} 1 & 0 \\ -1 & 0 \\ 1 & 0 \end{bmatrix}$$

$$\delta_1 = \left({W_2}^T e \right) \odot \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

$$\nabla_{W_1} L = \delta_1 x^T = \begin{bmatrix} 2 & 0 & -2 \\ 0 & 0 & 0 \end{bmatrix}$$

출력 가중치의 gradient는 오차와 은닉층 활성화 전치의 곱입니다. 은닉층 오차는 출력 가중치와 ReLU 미분을 거쳐 전달되며, 활성화 전 값이 음수인 성분에서는 gradient가 0이 됩니다.

Bias gradient는 해당 오차 벡터입니다. 학습률 0.1로 가중치와 bias를 모두 갱신하면 다음 예측은 [0.26, 0.14, 0.26]이고, loss ½‖예측 − target‖²는 1.5에서 0.4374로 줄어듭니다.

PyTorch autograd는 이 chain rule을 자동으로 계산합니다. Notebook에서 손계산과 대조합니다.

가중치 하나의 연쇄법칙

손계산 손실은 $L=\frac{1}{2}\sum e_j^2$이고 $e_j=\hat{y}_j-y_j$입니다. 따라서 출력 $\hat{y}_j$를 조금 늘렸을 때의 손실 변화율은 e_j입니다. 출력 가중치 $W_{2,ji}$를 늘리면 $\hat{y}_j$는 은닉값 h_i만큼 변합니다.

미분 경로	곱하는 값	앞 예제의 결과
$W_{2,A1} \to \hat{y}_A \to L$	$e_A \times h_1$	$1\times1=1$
$W_{2,B1} \to \hat{y}_B \to L$	$e_B \times h_1$	$-1\times1=-1$
$h_1 \to$ 세 출력 $\to L$	연결 가중치×오차의 합	$1\times1+0\times(-1)+1\times1=2$
$W_{1,13} \to a_1 \to h_1 \to L$	$x_3 \times \mathrm{ReLU}'(a_1) \times 2$	$(-1)\times1\times2=-2$

ReLU 미분은 $a>0$에서 1, $a<0$에서 0입니다. 이 예는 $a=0$을 피합니다. Bias는 가중합에 1배로 더해지므로 그 미분에는 입력값 대신 1을 곱합니다.

δ는 해당 층의 활성화 전 값에 대한 손실 미분, $\odot$는 같은 위치끼리의 곱입니다. 실제 평균제곱 손실에서는 출력 미분이 $2e/(3N)$이므로 손계산의 ½ 제곱합과 계수가 다릅니다.

가중치 갱신 뒤의 두 번째 순전파

학습률 0.1로 가중치와 편향을 모두 갱신합니다. 첫 은닉 가중치 [1,0,0]에서 0.1×[2,0,−2]를 빼면 [0.8,0,0.2], 편향은 −0.2가 됩니다.

중간값	갱신 뒤 계산
첫 은닉 pre-activation	0.8×1+0×0+0.2×(−1)−0.2=0.4
두 번째 은닉 pre-activation	−1
ReLU 출력	[0.4,0]
출력 A, C	0.9×0.4−0.1=0.26
출력 B	0.1×0.4+0.1=0.14

새 오차는 [0.26,−0.86,0.26]입니다. 손실은 (0.0676+0.7396+0.0676)/2=0.4374로 줄지만 B는 여전히 목표 1보다 작습니다. 한 번의 감소가 학습 완료를 뜻하지는 않습니다.

5. 학습 loop를 계산 순서로 읽기

한 batch에서 예측과 loss를 계산한 뒤, 이전에 누적된 파라미터 gradient를 지웁니다. 이어 현재 loss를 미분하고 optimizer로 파라미터를 갱신합니다.

```
prediction = model(inputs)
loss = loss_function(prediction, targets)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

`loss.backward()`는 gradient를 계산하고 `optimizer.step()`은 파라미터를 바꿉니다. Gradient를 지우지 않으면 다음 backward 결과가 기존 값에 누적됩니다.

Epoch 뒤 validation loss를 평가해 가장 작은 값의 가중치를 보관합니다. Test는 학습과 모델 선택에 사용하지 않으며, 예측 지표는 출력 표준화를 되돌린 후 계산합니다.

구현 설명은 PyTorch의 학습 loop, 모델 구성을 참고합니다.

학습 루프의 값과 변경 시점

```
prediction = model(inputs)
loss = loss_function(prediction, targets)
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

줄	만들어지거나 바뀌는 값
model(inputs)	현재 가중치로 batch의 출력 계산
loss_function	출력과 정답 차이를 스칼라 손실로 축약
zero_grad	이전 batch의 기울기 제거
backward	현재 손실의 가중치·편향별 기울기
step	기울기와 optimizer 상태로 파라미터 갱신

손 계산은 단순 gradient descent였고 실제 학습은 Adam입니다. Adam은 과거 기울기 정보도 사용하므로 같은 0.1배 빼기와 수치가 같지 않습니다. step 뒤의 성능을 보려면 갱신된 가중치로 순전파를 다시 수행합니다.

Adam의 첫 갱신과 batch의 의미

Batch는 한 번에 손실과 기울기를 계산하는 데이터 묶음입니다. 400개 행을 batch 64로 나누면 한 epoch에 64개짜리 6묶음과 16개짜리 마지막 묶음, 총 7번 갱신합니다. Epoch는 학습 데이터 한 바퀴입니다.

Adam은 기울기 평균 m과 제곱 평균 v를 누적합니다. 처음 $m=v=0$, 기울기 $g=2$, $\beta_1=0.9$, $\beta_2=0.999$인 가중치 하나를 계산합니다.

계산	첫 단계 값
$m=0.9m+0.1g$	0.2
$v=0.999v+0.001g^2$	0.004
초기 0의 편향 보정	$\hat{m}=0.2/(1-0.9)=2$; $\hat{v}=0.004/(1-0.999)=4$
학습률 0.001의 이동 크기	$0.001\times2/(\sqrt{4}+10^{-8})\approx0.001$

단순 경사하강이라면 $0.001\times2=0.002$를 뺍니다. Adam은 누적된 기울기 크기로 이동을 조정하므로 같은 학습률이어도 갱신량이 다릅니다. 이 계산의 $\varepsilon=10^{-8}$은 분모가 0이 되는 것을 막습니다.

PyTorch Adam의 갱신 정의

6. 독립 예측을 진단하기

기준값이 [0.6, 0.4, 0.2] mol/L이고 예측 모델이 [0.62, 0.39, 0.19] mol/L를 반환한다고 가정합니다.

성분별 오차는 [0.02, −0.01, −0.01] mol/L입니다. 두 농도 합은 모두 1.2이므로 합 residual은 0이지만 각 성분의 오차는 남아 있습니다.

Test 보고서에는 어떤 지표를 넣어야 할까요? 모델 선택에는 어떤 데이터를 사용해야 할까요?

답 확인: 물리 단위의 A·B·C RMSE, 농도 합 residual과 최소 예측 농도를 보고합니다. Train으로 학습하고 validation으로 선택하며 test는 두 단계에 사용하지 않습니다.

2주차 필수 실습에서 같은 지표를 확인합니다.

표준화 출력에서 농도와 오차로

계산용 한 성분의 학습 평균 μ=0.4 mol/L, 표준편차 s=0.1 mol/L를 둡니다. 정답 농도 0.45는 z=(0.45−0.4)/0.1=0.5입니다.

항목	계산
신경망의 표준화 예측	$\hat{z}=0.3$
농도 복원	0.4+0.1×0.3=0.43 mol/L
표준화 오차	0.3−0.5=−0.2
농도 오차	0.43−0.45=−0.02 mol/L

```
prediction = y_mean + y_scale * standardized_prediction
component_rmse = np.sqrt(((prediction - truth)**2).mean(axis=0))
sum_residual = prediction.sum(axis=1) - feed
```

첫 줄은 각 출력 열에 해당 평균과 표준편차를 적용합니다. axis=0은 성분별 정확도, axis=1은 같은 조건의 세 성분 합을 계산합니다. 두 축을 바꾸면 다른 질문을 계산하게 됩니다.